GridPulse: Crypto-Induced Latency Denial of Service in IEC 62351-6 Substation Protection Communications

A Complete Technical Analysis with Mathematical Proofs, Hardware Evidence, and Kernel-Level Defense

Dr. Sapna Vikram Mewundi* **Prajwal Chowdary M.†**

Prof. Prasad B. Honnavalli‡

October 2026

Abstract

We present **Crypto-Induced Latency Denial of Service (CIL-DoS)**, a novel protocol-level vulnerability in IEC 62351-6, the cryptographic authentication standard for IEC 61850 GOOSE messages in electrical substations. The standard mandates that Intelligent Electronic Devices (IEDs) verify every incoming frame's cryptographic signature before acceptance. We prove mathematically, and validate on real hardware, that this requirement creates a fundamental denial-of-service surface: an attacker flooding invalid-signature frames at rate λ saturates the verification queue when server utilization $\rho = \lambda/\mu > 1$, causing safety-critical trip commands to miss their 3.0 ms deadline. Our evidence spans ESP32 (Xtensa LX6, 240 MHz) and Raspberry Pi 5 (Cortex-A76, 2.4 GHz) platforms with 50,000-iteration benchmarks, 50,000-frame PCAP captures, 49 controlled experiments, and a novel XDP/eBPF kernel-level defense (PQAC) achieving 99% attack frame rejection at the NIC driver layer. We propose PQAC for standardization in future IEC 62351 revisions and have initiated coordinated disclosure via CISA VINCE-NT.

Keywords: IEC 62351-6, IEC 61850, GOOSE, CIL-DoS, denial of service, cryptographic verification, queueing theory, XDP/eBPF, substation protection, CVE, CWE-400

1 Introduction: The Problem

1.1 What is a Substation?

An electrical substation transforms voltage levels and routes power. Inside it, **protection relays** monitor for faults (short circuits, overcurrent) and send **trip commands** to circuit breakers to isolate the fault before equipment is damaged.

Key Insight

If a trip command is delayed, the fault current continues flowing. A 345 kV transformer carrying 10× rated current accumulates thermal energy at $I^2t = 100 \times t$ — at this rate, winding insulation fails in ∼2 seconds. The relay has **3 milliseconds** to deliver the trip.

1.2 How Relays Communicate: IEC 61850 GOOSE

Modern substations replaced copper wiring with Ethernet. IEC 61850 defines the **GOOSE** (Generic Object Oriented Substation Event) protocol:

- **Layer 2** — raw Ethernet, EtherType `0x88B8`
- **Multicast** — one publisher, many subscribers
- **No IP/TCP** — no routing, no retransmission
- **3 ms deadline** — IEC 61850-5, Performance Class P2/P3

A GOOSE frame carries ASN.1 BER-encoded fields:

- `gocbRef`: control block reference (identifies the publisher)
- `stNum`: state number (increments on each data change)
- `sqNum`: sequence number (increments within a state)
- `datSet`: the actual protection data (trip/close)

*Associate Professor, Dept. of CSE, PES University, Bengaluru. Email: sapnavm@pes.edu

†Security Researcher, PES University. Email: chowdaryprajwal2@gmail.com

‡Professor, Dept. of CSE, PES University, Bengaluru.

1.3 The Security Problem: Spoofing

Without authentication, any device on the Layer-2 network can forge GOOSE frames — sending fake trip commands or suppressing real ones. This is well-documented [1, 2].

1.4 The Security "Fix": IEC 62351-6

IEC 62351-6:2020 adds cryptographic authentication to GOOSE. Each frame gets a security extension (ASN.1 tag `0xAF`) containing a digital signature or MAC:

- **HMAC-SHA-256**: symmetric, fast, requires shared keys
- **ECDSA-P256**: asymmetric, slower, no key sharing needed
- **RSA-2048**: asymmetric, moderate speed, large signatures

Attack Principle

The standard says: *"The receiving entity shall verify the authentication value of every received GOOSE message before processing."* This single sentence is the root cause of CIL-DoS.

2 The Vulnerability: Step by Step

2.1 Step 1: Understand the Asymmetry

Definition 1 (Computational Asymmetry)**.** *Let t_{gen} be the time to* generate *a GOOSE frame with a random (invalid) signature, and t_{verify} be the time to* verify *that signature. For all public-key schemes:*

$$t_{gen} \ll t_{verify} \tag{1}$$

Generating a frame with random bytes in the signature field costs $\sim 1\,\mu s$ (memory copy). Verifying RSA-2048 costs $\sim 100\,\mu s$ (modular exponentiation). The ratio is $\sim$100:1.

Why this matters: The attacker does almost no work per frame. The defender must do expensive math on *every single frame* — even the invalid ones — because there is no way to know a signature is invalid without checking it.

2.2 Step 2: The Queueing Model

Model the IED's verification subsystem as a queue:

Definition 2 (M/M/1 Queue)**.** *A single-server queue where:*

- *Arrivals follow a Poisson process with rate λ (packets/sec)*
- *Service times are exponentially distributed with rate $\mu = 1/\bar{t}_{verify}$*
- *One CPU core processes verifications sequentially (FIFO)*

The total arrival rate during an attack is:

$$\lambda_{\text{total}} = \lambda_{\text{legit}} + \lambda_{\text{attack}} \tag{2}$$

Since $\lambda_{\text{attack}} \gg \lambda_{\text{legit}}$, we approximate $\lambda \approx \lambda_{\text{attack}}$.

Definition 3 (Server Utilization)**.**

$$\boxed{\rho = \frac{\lambda}{\mu} = \lambda \cdot \bar{t}_{verify}} \tag{3}$$

ρ is the fraction of time the server is busy. When $\rho \geq 1$, the server cannot keep up — frames arrive faster than they can be verified.

2.3 Step 3: What Happens When $\rho > 1$

Theorem 1 (Queue Instability)**.** *For an M/M/1 queue with $\rho \geq 1$, the queue length $L(t) \to \infty$ as $t \to \infty$. Specifically, the queue grows at rate:*

$$\frac{dL}{dt} = \lambda - \mu = \mu(\rho - 1) \tag{4}$$

The expected waiting time $W_q \to \infty$.

Proof. From Burke's theorem and the balance equations of an M/M/1 queue, the steady-state distribution exists only when $\rho < 1$:

$$P(N = n) = (1 - \rho)\rho^n, \quad \rho < 1 \tag{5}$$

where N is the number in system. The expected number in queue is:

$$L_q = \frac{\rho^2}{1 - \rho} \tag{6}$$

As $\rho \to 1^-$, $L_q \to \infty$. For $\rho \geq 1$, no steady state exists. The queue length is a random walk with positive drift $\lambda - \mu > 0$, so $L(t) \to \infty$ a.s. by the strong law of large numbers.

Concretely: if $\lambda = 10{,}000$ pps and $\mu = 5{,}164$ pps (ECDSA on Pi 5), then in one second, $10{,}000 - 5{,}164 = 4{,}836$ frames accumulate in the queue. After 10 seconds: 48,360 frames waiting. A legitimate trip command arriving at time t finds $\sim 4{,}836t$ frames ahead of it. □

2.4 Step 4: The Trip Delay Formula

Theorem 2 (Trip Command Delay Under Attack)**.** *A legitimate trip command arriving at time t after the attack begins experiences a waiting time of approximately:*

$$\boxed{W_{trip}(t) \approx (\rho - 1) \cdot \mu^{-1} \cdot \lambda \cdot t = (\lambda - \mu) \cdot t \cdot \bar{t}_{verify}} \tag{7}$$

For the trip to arrive within the 3 ms deadline:

$$W_{trip}(t) \leq 3 \times 10^{-3} \text{ seconds} \tag{8}$$

Proof. At time t, the queue has accumulated $\approx (\lambda - \mu) \cdot t$ frames (by the drift rate from Theorem 1). Each frame ahead of the trip command takes $\bar{t}_{\text{verify}} = \mu^{-1}$ seconds to process. Therefore:

$$W_{\text{trip}}(t) \approx [(\lambda - \mu) \cdot t] \cdot \mu^{-1} \tag{9}$$

$$= \frac{(\lambda - \mu) \cdot t}{\mu} \tag{10}$$

Example: $\lambda = 100{,}000$ pps, ECDSA-P256 on Pi 5 ($\mu = 5{,}164$ pps, $\bar{t}_{\text{verify}} = 193.63\,\mu$s). After just $t = 0.1$ seconds:

$$W_{\text{trip}} \approx (100{,}000 - 5{,}164) \times 0.1 \times 193.63 \times 10^{-6} \tag{11}$$

$$= 94{,}836 \times 0.1 \times 1.9363 \times 10^{-4} \tag{12}$$

$$= 1.836 \text{ seconds} \tag{13}$$

This is **612**× past the 3 ms deadline after just 100 ms of attack. □

2.5 Step 5: Why the Attacker Needs No Keys

Proposition 1 (Key-Free Attack)**.** *The CIL-DoS attack requires zero cryptographic keys. The attacker's frames contain random bytes in the signature field.*

Proof. The verification algorithm must execute completely before it can determine whether a signature is valid or invalid:

RSA-2048 verification: compute $s^e \bmod n$ where s is the signature, e is the public exponent, n is the modulus. This requires modular exponentiation regardless of whether s is valid. The result is compared to the hash of the message only *after* the exponentiation completes.

ECDSA-P256 verification: compute $u_1 \cdot G + u_2 \cdot Q$ where G is the generator point, Q is the public key, and u_1, u_2 are derived from the signature and message hash. The elliptic curve point multiplications execute regardless of signature validity.

In both cases, the CPU cost is paid *before* the validity decision. Random bytes trigger the same computation as a real signature — the IED cannot short-circuit. □

Key Insight

This is the fundamental insight: **invalid signatures cost the same to verify as valid ones**. The attacker weaponizes the defender's own security mechanism.

3 Hardware Evidence

3.1 Why Real Hardware Matters

Prior work [3, 4] benchmarked on server-class CPUs. Real IEDs use embedded processors (ARM Cortex-M/R at 200–800 MHz, or Xtensa/RISC-V at 80–400 MHz). We benchmark on two platforms that bracket real IED hardware.

3.2 Platform 1: ESP32 (Embedded Class)

Hardware: Espressif ESP32-D0WDQ6, Xtensa LX6 dual-core @ 240 MHz, 520 KB SRAM. Represents low-end IED processors.

Table 1: ESP32 cryptographic verification benchmarks

Algorithm	Mean (μs)	μ (vps)	ρ @ 10K
HMAC-SHA-256	167.3	5,978	1.67
RSA-2048	980.1	1,020	9.80
ECDSA-P256	310,475.9	3.2	3,125

Attack Principle

A single ECDSA-P256 verification on ESP32 takes **310 ms** — that's 103× past the 3 ms deadline *without any attack*. The standard's recommended algorithm is physically impossible on embedded hardware.

The math: At 10,000 pps with ECDSA on ESP32:

$$\rho = \frac{10{,}000}{3.2} = 3{,}125 \tag{14}$$

The IED can verify 3.2 frames per second while 10,000 arrive. After 1 second, the queue has ∼9,997 frames. A trip command waits:

$$W = 9{,}997 \times 0.310 = 3{,}099 \text{ seconds} \approx \textbf{52 minutes} \tag{15}$$

3.3 Platform 2: Raspberry Pi 5 (Server Class)

Hardware: BCM2712, ARM Cortex-A76 @ 2.4 GHz, 16 GB RAM, ARM crypto extensions, OpenSSL 3.5.7. This is **much faster** than any production IED — it represents the absolute best case.

Benchmark methodology: 50,000 iterations per algorithm. We report mean, standard deviation, percentiles, and derive the saturation rate $\mu = 1/\bar{t}_{\text{verify}}$.

Table 2: Pi 5 Cortex-A76 benchmarks (50K iterations)

Algorithm	Mean (μs)	P99 (μs)	μ (vps)	ρ @ 10K
HMAC-SHA-256	2.33	2.35	429,185	0.023
RSA-2048	100.85	102.90	9,916	1.01
Ed25519	143.98	146.07	6,945	1.44
ECDSA-P256	193.63	288.02	5,164	1.94

Key finding: The Pi 5 is **1,603× faster** than ESP32 for ECDSA. Yet even here:

- RSA-2048 saturates at exactly 10K pps ($\rho = 1.01$)
- ECDSA saturates at just 5,164 pps ($\rho = 1.94$ at 10K)
- Only HMAC-SHA-256 survives ($\rho = 0.023$)

3.4 The Saturation Threshold

Definition 4 (Saturation Rate)**.** *The maximum attack rate an IED can withstand without queue divergence:*

$$\boxed{\lambda_{sat} = \mu = \frac{1}{\bar{t}_{verify}}} \tag{16}$$

Any flood rate $\lambda > \lambda_{sat}$ causes unbounded delay.

Table 3: ρ values across platforms and attack rates

Platform	Algorithm	ρ@1K	ρ@10K	ρ@100K
ESP32	HMAC-SHA-256	0.17	1.67	16.7
ESP32	RSA-2048	0.98	9.80	98.0
ESP32	ECDSA-P256	312.5	3,125	31,250
Pi 5	HMAC-SHA-256	0.002	0.023	0.23
Pi 5	RSA-2048	0.10	1.01	10.1
Pi 5	Ed25519	0.14	1.44	14.4
Pi 5	ECDSA-P256	0.19	1.94	19.4

Key Insight

The table tells the whole story: **red cells ($\rho > 1$) mean the attack succeeds**. On ESP32, everything except HMAC at very low rates is red. On Pi 5, every asymmetric algorithm goes red at 10K pps — a rate trivially achievable on commodity hardware.

4 Why Naive Defenses Fail

4.1 "Just Use a Bigger Buffer"

Theorem 3 (Buffer Overflow Time)**.** *A queue of finite size B overflows in:*

$$T_{overflow} = \frac{B}{\lambda - \mu} \tag{17}$$

Proof. When $\rho > 1$, the net arrival rate exceeds service: frames accumulate at rate $\lambda - \mu$. A buffer of B frames fills in $B/(\lambda - \mu)$ seconds.

Example: $B = 10{,}000$ frames, $\lambda = 100{,}000$ pps, $\mu = 5{,}164$ (ECDSA/Pi 5):

$$T_{\text{overflow}} = \frac{10{,}000}{100{,}000 - 5{,}164} = \frac{10{,}000}{94{,}836} = 0.105 \text{ sec} \tag{18}$$

The buffer overflows in 105 ms. After that, frames are dropped — including legitimate trip commands.

Worse: even before overflow, the trip command at the back of a 10,000-frame queue waits:

$$W = 10{,}000 \times 193.63 \times 10^{-6} = 1.936 \text{ seconds} \tag{19}$$

A bigger buffer *increases* trip delay. It makes the problem worse. □

4.2 "Just Add More CPU Cores"

Theorem 4 (Multi-Core Requirement)**.** *An M/M/c queue (with c parallel verification cores) requires:*

$$c > \frac{\lambda}{\mu} = \rho_1 \tag{20}$$

cores to achieve stability.

Proof. For an M/M/c queue, stability requires $\rho_c = \lambda/(c \cdot \mu) < 1$, i.e., $c > \lambda/\mu$.

At 100K pps with ECDSA on Pi 5: $c > 100{,}000/5{,}164 = 19.4$, requiring **20 dedicated CPU cores** for verification alone. IED hardware typically has 1–4 cores total. □

4.3 "Just Use Faster Crypto"

Proposition 2 (Arms Race Futility)**.** *If verification speed improves by factor k, the attacker increases flood rate by k to restore $\rho > 1$. Frame generation cost is near-zero and limited only by link bandwidth.*

Proof. Suppose the IED verifies k times faster: $\mu' = k\mu$. The attacker floods at $\lambda' = k\lambda$. Then:

$$\rho' = \frac{k\lambda}{k\mu} = \frac{\lambda}{\mu} = \rho \tag{21}$$

Bandwidth limit: a minimal GOOSE frame is ∼154 bytes. On a 1 Gbps process bus:

$$\lambda_{\max} = \frac{10^9}{154 \times 8} = 811{,}688 \text{ pps} \tag{22}$$

This is 157× the ECDSA saturation rate on Pi 5. The attacker has enormous headroom. □

5 PCAP Evidence on Real Hardware

5.1 Testbed Architecture

We built a Layer-2 testbed on Raspberry Pi 5 using a Linux virtual bridge (`goose-br`) with veth pairs:

- `veth-pub`: publisher sends frames (our C generator)
- `goose-br`: bridge forwards frames (simulates process bus)
- `veth-sub`: subscriber receives + tcpdump captures

5.2 Frame Generator: `goose_pcap_gen.c`

Our C program generates wire-format GOOSE frames using raw sockets (`AF_PACKET`). Each frame contains:

- Ethernet header with EtherType `0x88B8`
- GOOSE header: APPID (2B) + Length (2B) + Reserved (4B)
- ASN.1 BER-encoded PDU: `0x61` SEQUENCE with `gocbRef` (tag `0x80`), `timeAllowedtoLive` (`0x81`), `datSet` (`0x82`), `stNum` (`0x83`), `sqNum` (`0x84`), `confRev` (`0x85`)
- IEC 62351-6 security extension (tag `0xAF`): algorithm ID + 32-byte signature

Three modes:

1. **Legit:** src MAC `08:00:27:10:20:01`, signature `0xAA` repeated
2. **Attack:** src MAC `de:ad:be:ef:00:01`, random signature bytes
3. **Mixed:** 1 legit per 99 attack frames

5.3 Capture Results

Table 4: PCAP datasets captured on Pi 5

File	Legit	Attack	Total	Size
`goose_legit_pi5.pcap`	100	0	100	15 KB
`goose_cildos_mixed_pi5.pcap`	500	49,500	50,000	7.7 MB

Sustained flood rate: **6,423 pps** — exceeding ECDSA saturation (5,164 pps) on Pi 5 hardware. All frames verified: correct EtherType, proper MAC separation, valid ASN.1 structure.

6 Physical Consequences

6.1 Generator Rotor Instability

When a fault occurs, the generator accelerates because input mechanical power exceeds output electrical power. The **swing equation**:

$$M\frac{d^2\delta}{dt^2} = P_m - P_e(\delta) \tag{23}$$

where M is inertia, δ is rotor angle, P_m is mechanical power, $P_e(\delta) = P_{\max}\sin\delta$ is electrical power.

The **Critical Clearing Time** t_{crit} is the maximum time to clear the fault before the rotor angle exceeds the point of no return (transient stability limit). Typical values:

- Conventional generators: $t_{\text{crit}} \approx 150$–$300$ ms
- Low-inertia renewables: $t_{\text{crit}} \approx 80$–$150$ ms

CIL-DoS causes trip delays of >2,000 ms — far past any t_{crit}.

6.2 Transformer Thermal Damage

Through-fault thermal energy follows:

$$E = I^2 \cdot R \cdot t \tag{24}$$

At 6.25× rated current (from a fault with delayed clearing):

$$E = (6.25)^2 \cdot R \cdot t = 39.06 \cdot R \cdot t \tag{25}$$

Per IEEE C57.109, transformer damage curves show winding insulation failure begins at ∼2 seconds of sustained fault current. CIL-DoS provides more than enough delay.

6.3 Cascading Failure

1. CIL-DoS delays trip at Substation A
2. Fault current flows to adjacent lines
3. Distance relays at Substations B, C trip (correct operation)
4. Load redistribution overloads remaining lines
5. Cascade propagates through interconnected grid

This is the mechanism behind major blackouts (Northeast 2003, India 2012).

7 The PQAC Defense

Defense Principle

The key insight: **don't verify everything**. Triage frames with O(1) checks *before* burning CPU on cryptography. Only frames that pass cheap pre-filters reach the expensive verification stage.

7.1 Three-Stage Architecture

Algorithm 1 PQAC: Priority Queue Admission Controller

Require: Incoming GOOSE frame f
Ensure: Accept or Drop decision

Stage 1: MAC Allowlist ▷ O(1) hash lookup
if f.srcMAC $\notin$ AllowList **then**
  return Drop ▷ $\sim 0.1\,\mu$s
end if

Stage 2: stNum Plausibility ▷ O(1) arithmetic
$\Delta \leftarrow f$.stNum − LastStNum[f.srcMAC]
if $\Delta \neq 0$ **and** $|\Delta| > W$ **then** ▷ $W = 16$
  return Drop ▷ $\sim 0.05\,\mu$s
end if
LastStNum[f.srcMAC] $\leftarrow f$.stNum

Stage 3: Verify-Before-Commit ▷ Full crypto
if CryptoVerify(f) = Invalid **then**
  return Drop
end if
return Accept ▷ Execute trip command

Stage 1 — MAC Allowlist: The Substation Configuration Description (SCD file, per IEC 61850-6) lists every authorized IED's MAC address. We build a hash table at boot. Unknown MACs are dropped in $\sim 0.1\,\mu$s.

Why it works: The attacker's frames come from MAC `de:ad:be:ef:00:01` (or random MACs). These are not in the SCD. Stage 1 drops them instantly — no crypto needed.

Stage 2 — stNum Plausibility: GOOSE `stNum` increments monotonically (by 1 on each state change). We track the last accepted `stNum` per source and accept only values within a window $W = 16$.

Why it works: Attack frames have random or sequential `stNum` values. After the first few, subsequent values fall outside the plausibility window and are dropped.

Stage 3 — Verify-Before-Commit: Only frames passing Stages 1 and 2 undergo crypto verification. No state change occurs until verification succeeds.

7.2 XDP/eBPF Implementation

We implemented PQAC Stages 1–2 as an XDP (eXpress Data Path) program running at the **NIC driver level** in the Linux kernel — the earliest possible packet processing point.

What is XDP? XDP is a Linux kernel framework that lets eBPF programs process packets *before* they enter the network stack. Packets can be dropped at near-zero cost:

- **XDP_DROP**: discard packet at driver (never allocates `sk_buff`)
- **XDP_PASS**: forward to normal network stack
- **XDP_TX**: bounce back out the same interface

Our XDP program (`xdp_pqac.c`) uses three BPF maps:

1. `mac_allowlist`: BPF_MAP_TYPE_HASH, key = 6-byte MAC
2. `stnum_tracker`: BPF_MAP_TYPE_HASH, key = src MAC, value = last stNum
3. `pqac_stats`: BPF_MAP_TYPE_ARRAY, 5 counters

7.3 XDP Test Results on Pi 5

Table 5: XDP PQAC results (Pi 5, native mode, kernel 6.18.50)

Metric	Value
Total frames processed	50,101
Stage 1 drops (MAC allowlist)	49,500 (99.0%)
Stage 2 drops (stNum plausibility)	10
Passed to userspace	590
Attack frame drop rate	**99.0%**
XDP attach mode	native (not generic)
Flood rate	6,427 pps

Key Insight

49,500 out of 49,500 attack frames were dropped at Stage 1 (MAC allowlist) — **before any cryptographic verification**. The verification queue only sees 590 frames (the legitimate ones), keeping $\rho \ll 1$. Trip commands arrive in microseconds, not seconds.

7.4 Defense Effectiveness Proof

Theorem 5 (PQAC Reduces Effective ρ)**.** *Let p be the fraction of attack frames that pass PQAC Stages 1–2. The effective server utilization becomes:*

$$\rho_{PQAC} = \frac{\lambda_{legit} + p \cdot \lambda_{attack}}{\mu} \tag{26}$$

With $p < \mu/\lambda_{attack}$, stability is restored ($\rho_{PQAC} < 1$).

Proof. Without PQAC: $\rho = (\lambda_{\text{legit}} + \lambda_{\text{attack}})/\mu$.

With PQAC: only fraction p of attack frames reach crypto. The effective arrival rate at Stage 3 is $\lambda_{\text{legit}} + p \cdot \lambda_{\text{attack}}$.

For stability: $\rho_{\text{PQAC}} < 1$, i.e.:

$$p < \frac{\mu - \lambda_{\text{legit}}}{\lambda_{\text{attack}}} \tag{27}$$

Our measurement: $p = 0/49{,}500 = 0$ for unknown-MAC attacks. Even for MAC-spoofing attacks (where p is higher due to Stage 1 bypass), Stage 2 reduces throughput further.

Numeric example: $\lambda_{\text{legit}} = 10$ pps, $\lambda_{\text{attack}} = 100{,}000$ pps, $\mu = 5{,}164$ (ECDSA/Pi 5), $p = 0.001$ (0.1% pass rate):

$$\rho_{\text{PQAC}} = \frac{10 + 0.001 \times 100{,}000}{5{,}164} = \frac{110}{5{,}164} = 0.021 \tag{28}$$

From $\rho = 19.4$ to $\rho = 0.021$ — a **923× improvement** in utilization. □

8 Docker Testbed: 49 Experiments

8.1 Experimental Setup

Three Docker containers connected via a bridge network:

- **Publisher**: sends legitimate GOOSE trips at 1 pps
- **Attacker**: floods invalid GOOSE at configurable rate
- **Subscriber**: FIFO or PQAC mode, measures trip latency

Reproduce with one command:

```
docker compose run --rm subscriber \
  --mode fifo --attack-rate 100000
```

8.2 Key Results

Table 6: Trip latency: FIFO vs. PQAC (100K pps attack)

Metric	FIFO	PQAC
Mean trip latency	2,388 ms	0.76 ms
Max trip latency	3,847 ms	1.12 ms
Deadline miss rate (3 ms)	100%	0.00%
Improvement	—	3,140×

The 49 experiments varied: attack rate (1K/10K/50K/100K pps), algorithm (HMAC/ECDSA/RSA), defense mode (FIFO/PQAC), queue depth (100/1K/10K). All PQAC configurations achieved 0.00% deadline miss.

9 Adaptive Adversary Analysis

A sophisticated attacker might try to bypass PQAC:

Table 7: Evasion strategies vs. PQAC

Strategy	Bypasses	Stopped by	Result
Random MAC	—	Stage 1	Dropped
Spoofed MAC	Stage 1	Stage 2	Mostly dropped
MAC + stNum walk	Stage 1+2	Stage 3	Crypto load

Theorem 6 (Stage 3 Unforgeable Guarantee)**.** *Even if the attacker bypasses Stages 1 and 2, no state change occurs without a valid cryptographic signature. Without the private key, the probability of forging a valid signature is negligible:*

$$P(\textit{forge}) \leq \frac{1}{2^{128}} \approx 2.9 \times 10^{-39} \tag{29}$$

for 128-bit security level algorithms.

The adaptive attacker can increase verification load (a resource exhaustion), but cannot:

- Cause a *false* trip (integrity preserved)
- Prevent a legitimate trip from being *accepted* (PQAC keeps ρ low)

10 Post-Quantum Impact

NIST Post-Quantum standards (FIPS 204/205) would worsen CIL-DoS:

Table 8: Post-quantum signature impact

Algorithm	Sig. Size	Verify (μs)	MTU?
ECDSA-P256	64 B	194	OK
ML-DSA-44	2,420 B	~250	Exceeds
SLH-DSA-128s	7,856 B	~597,000	Exceeds

SLH-DSA verification at 597 ms is **199**× past the 3 ms deadline on a server CPU. Combined with signature sizes exceeding Ethernet MTU (requiring fragmentation), PQC makes CIL-DoS catastrophically worse without admission control.

11 CVE Justification

11.1 Why This Deserves a CVE

1. Protocol design flaw, not implementation bug: The vulnerability is in IEC 62351-6's requirement to verify every frame. All compliant implementations are affected.

2. Precedent — KRACK (CVE-2017-13077): WPA2's key reinstallation vulnerability was also a protocol design flaw in an international standard (IEEE 802.11i). KRACK was assigned CVEs through CERT/CC (CISA's predecessor). Our submission follows the same path.

3. Physical safety impact: Unlike most CVEs, CIL-DoS has consequences beyond the digital domain: generator instability, transformer damage, cascading blackouts.

4. CWE classification: CWE-400 (Uncontrolled Resource Consumption) + CWE-770 (Allocation Without Limits).

5. CVSS 3.1 score: 7.4 (High):

`CVSS:3.1/AV:A/AC:L/PR:N/UI:N/S:C/C:N/I:N/A:H`

11.2 Submission Route

CISA is the **Root CNA for Industrial Control Systems**. IEC is not a CNA and cannot assign CVE IDs. We submit through CISA VINCE-NT (`https://vulnerabilities.cisa.gov`), the same portal used for ICS vulnerabilities.

11.3 No Prior CVE Exists

We searched extensively: existing CVEs in the IEC 61850 space (CVE-2026-66369, CVE-2026-52134, CVE-2025-1445) are all **implementation bugs** in specific libraries (libiec61850, Hitachi RTU500). No CVE targets the IEC 62351-6 *standard's design* as a vulnerability.

11.4 Disclosure Timeline

Date	Action
August 2026	Vulnerability discovered
October 7, 2026	IEEE TSG manuscript submitted
October 22, 2026	Pi 5 hardware evidence collected
October 23, 2026	XDP/eBPF defense validated
October 2026	CISA VINCE-NT submission
October 2026	IEC TC 57 WG 15 notification
March 2027	Coordinated disclosure (BH Asia 2027)

12 Related Work

GOOSE spoofing attacks: Hoyos et al. [1] demonstrated spoofed GOOSE frames causing relay trips and blackouts using RTDS hardware-in-the-loop. Their work targets *unauthenticated* GOOSE — CIL-DoS targets the *authentication mechanism itself.*

GOOSE vulnerability analysis: Kabir-Querrec et al. [2] catalogued GOOSE attack types. They note flooding but do not connect it to crypto verification cost.

IEC 62351-6 performance: Hussain et al. [4] measured RSASSA-PSS at 5.45–6.38 ms on server CPUs, already past the deadline. Elbez et al. [3] propose anomaly detection for GOOSE DoS but focus on detection, not prevention.

ICS DoS: Yang et al. [5] demonstrated DoS on IEC 61850 substations. SEL [6] acknowledges GOOSE cybersecurity gaps.

Our contribution: First to (1) name and formalize CIL-DoS as a protocol-level vulnerability, (2) benchmark on real embedded hardware, (3) provide queueing-theoretic proof, (4) build and validate a kernel-level defense, (5) demonstrate the full cyber-to-physical failure chain.

13 Conclusion

IEC 62351-6's mandatory verification creates a fundamental vulnerability: the security mecha-

nism intended to protect substation communications can be weaponized to defeat the safety function it guards. We have proven this mathematically ($\rho > 1 \Rightarrow W_q \to \infty$), validated it on real hardware (ESP32 and Pi 5), captured wire-format evidence (50K PCAP frames), and demonstrated a working defense (PQAC/XDP with 99% attack rejection at the kernel layer).

We recommend that IEC TC 57 WG 15:

1. Mandate admission control (PQAC-style triage) in future IEC 62351 revisions
2. Deprecate asymmetric signatures for time-critical GOOSE
3. Define maximum verification latency budgets per Performance Class
4. Standardize XDP/eBPF-based packet filtering for IED platforms

All code, benchmarks, PCAPs, and the Docker testbed are open-source at: https://github.com/sapnadfir-blip/gridpulse

Acknowledgments

The authors thank PES University for laboratory access and computing resources.

References

[1] J. Hoyos, M. Dehus, and T. Brown, "Cyber attacks on protective relays in digital substations and impact analysis," *IEEE Trans. Ind. Informatics*, vol. 16, no. 12, pp. 7575–7584, Dec. 2020.

[2] A. Kabir-Querrec et al., "Vulnerability and impact analysis of the IEC 61850 GOOSE protocol in the smart grid," *Sensors*, vol. 21, no. 4, p. 1554, Feb. 2021.

[3] G. Elbez et al., "Authentication of GOOSE messages under timing constraints," KIT Karlsruhe, Tech. Rep., 2020.

[4] S. M. S. Hussain et al., "Performance evaluation and analysis of IEC 62351-6 probabilistic signature scheme for securing GOOSE messages," *IEEE Access*, vol. 7, pp. 32426–32439, 2019.

[5] Y. Yang et al., "Denial-of-service attack on IEC 61850-based substation automation system: a crucial cyber threat towards smart substation pathways," *Sensors*, vol. 21, no. 19, p. 6415, Sep. 2021.

[6] Schweitzer Engineering Laboratories, "IEC 61850 network cybersecurity: mitigating GOOSE message vulnerabilities," SEL Tech. Paper 6921, 2019.

[7] M. Vanhoef and F. Piessens, "Key reinstallation attacks: forcing nonce reuse in WPA2," in *Proc. CCS*, 2017, pp. 1313–1328.

[8] A. Khan et al., "Securing IEC 61850 systems using trusted execution environments and IEC 62351-6 integration," *Procedia Computer Science*, 2026.